

```

/**
 * KilimoPRO 2.0 - API Layer
 *
 * Complete REST API implementation for the agricultural intelligence platform
 */

import { NextResponse } from 'next/server';
import { z } from 'zod';
import { DataAggregator, IGAD } from '@lib/data';

//
=====
====
// 1. MARKET PRICES API
//
=====
====

// app/api/prices/route.ts

const PriceQuerySchema = z.object({
  country: z.string().length(2).optional(),
  crop: z.string().optional(),
  year: z.coerce.number().optional(),
  limit: z.coerce.number().default(10).min(1).max(100),
  format: z.enum(['json', 'csv', 'sms']).default('json'),
});

/**
 * GET /api/prices
 * Get market prices for crops
 *
 * Query Parameters:
 * - country: 2-letter country code (KE, ET, UG, etc.)
 * - crop: Crop code or name (maize, wheat, sorghum, etc.)
 * - year: Year to fetch prices for
 * - limit: Maximum number of results
 * - format: Response format (json, csv, sms)
 *
 * Example:
 * /api/prices?country=KE&crop=maize
 * /api/prices?country=KE
 * /api/prices?crop=maize
 */

```

```

export async function GET_Prices(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = PriceQuerySchema.safeParse({
    country: searchParams.get('country'),
    crop: searchParams.get('crop'),
    year: searchParams.get('year'),
    limit: searchParams.get('limit'),
    format: searchParams.get('format'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }

  try {
    let data;

    if (query.data.country) {
      // Get prices for specific country
      data = await DataAggregator.getMarketPrices(query.data.country, query.data.crop);
    } else if (query.data.crop) {
      // Get prices for specific crop across all countries
      const countries = Object.keys(IGAD.COUNTRIES);
      data = await Promise.all(
        countries.map(c => DataAggregator.getMarketPrices(c, query.data.crop))
      );
      // Flatten the array
      data = data.flat();
    } else {
      // Get all prices for all countries
      const countries = Object.keys(IGAD.COUNTRIES);
      data = await Promise.all(
        countries.map(c => DataAggregator.getMarketPrices(c))
      );
      data = data.flat();
    }

    // Apply limit
    if (query.data.limit && Array.isArray(data)) {
      data = data.slice(0, query.data.limit);
    }
  }
}

```

```

}

// Format response based on format parameter
switch (query.data.format) {
  case 'csv':
    const csv = this.formatPricesAsCSV(data as any[]);
    return new NextResponse(csv, {
      headers: {
        'Content-Type': 'text/csv',
        'Content-Disposition': 'attachment; filename="market_prices.csv"'
      }
    });

  case 'sms':
    const sms = this.formatPricesAsSMS(data as any[]);
    return NextResponse.json({ success: true, message: sms });

  default:
    return NextResponse.json({
      success: true,
      data,
      count: Array.isArray(data) ? data.length : 1,
      timestamp: new Date().toISOString()
    });
}
} catch (error) {
  console.error('Market Prices API Error:', error);
  return NextResponse.json(
    { success: false, error: 'Failed to fetch market prices' },
    { status: 500 }
  );
}
}

function formatPricesAsCSV(prices: any[]): string {
  if (!prices || prices.length === 0) return 'No data available';

  const headers = ['Country', 'Crop', 'Price', 'Currency', 'Unit', 'Date', 'Source'];
  const rows = prices.map(p => [
    p.country || p.countryCode || "",
    p.crop || p.item || "",
    p.price || p.value || "",
    p.currency || "",
    p.unit || "",
  ]);

```

```

    p.date || p.year || ",
    p.source || "
  ].join(',');

  return [headers.join(','), ...rows].join('\n');
}

function formatPricesAsSMS(prices: any[]): string {
  if (!prices || prices.length === 0) return 'No price data available';

  // SMS has 160 character limit, so we need to be concise
  const first = prices[0];
  return `${first.crop || first.item} in ${first.country || first.countryCode}: ${first.price}
  ${first.currency}/${first.unit}`;
}

//
=====
====
// 2. CLIMATE ALERTS API
//
=====
=====

// app/api/climate/alerts/route.ts

const AlertsQuerySchema = z.object({
  country: z.string().length(2).optional(),
  type: z.enum(['drought', 'flood', 'pest', 'rainfall', 'extreme_rainfall', 'locust']).optional(),
  severity: z.enum(['low', 'moderate', 'high', 'extreme']).optional(),
  active: z.coerce.boolean().default(true),
  limit: z.coerce.number().default(10).min(1).max(50),
});

/**
 * GET /api/climate/alerts
 * Get hazard alerts from ICPAC
 *
 * Query Parameters:
 * - country: 2-letter country code
 * - type: Alert type (drought, flood, pest, etc.)
 * - severity: Alert severity (low, moderate, high, extreme)
 * - active: Only active alerts
 * - limit: Maximum number of results

```

*

* Example:

* /api/climate/alerts?country=KE

* /api/climate/alerts?type=drought&severity=high

*/

```
export async function GET_ClimateAlerts(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = AlertsQuerySchema.safeParse({
    country: searchParams.get('country'),
    type: searchParams.get('type'),
    severity: searchParams.get('severity'),
    active: searchParams.get('active'),
    limit: searchParams.get('limit'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }

  try {
    let alerts = await DataAggregator.getHazardAlerts(query.data.country);

    // Filter by type
    if (query.data.type) {
      alerts = alerts.filter(a => a.type === query.data.type);
    }

    // Filter by severity
    if (query.data.severity) {
      alerts = alerts.filter(a => a.severity === query.data.severity);
    }

    // Filter by active status
    if (query.data.active) {
      alerts = alerts.filter(a => {
        if (!a.endDate) return true; // No end date means still active
        const endDate = new Date(a.endDate);
        return endDate >= new Date();
      });
    }
  }
}
```

```

// Apply limit
if (query.data.limit) {
  alerts = alerts.slice(0, query.data.limit);
}

return NextResponse.json({
  success: true,
  alerts,
  count: alerts.length,
  timestamp: new Date().toISOString()
});
} catch (error) {
  console.error('Climate Alerts API Error:', error);
  return NextResponse.json(
    { success: false, error: 'Failed to fetch climate alerts' },
    { status: 500 }
  );
}
}

//
=====
====
// 3. AGRICULTURE WATCH API
//
=====
=====

// app/api/climate/watch/route.ts

const WatchQuerySchema = z.object({
  date: z.string().optional(),
  summary: z.coerce.boolean().default(false),
});

/**
 * GET /api/climate/watch
 * Get Agriculture Watch data from ICPAC
 *
 * Query Parameters:
 * - date: Specific date to fetch
 * - summary: Only return summary
 *

```

```

* Example:
* /api/climate/watch
* /api/climate/watch?summary=true
*/
export async function GET_AgricultureWatch(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = WatchQuerySchema.safeParse({
    date: searchParams.get('date'),
    summary: searchParams.get('summary'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }

  try {
    const watch = await DataAggregator.getAgricultureWatch();

    if (query.data.summary) {
      return NextResponse.json({
        success: true,
        summary: watch.summary,
        lastUpdated: watch.date,
        timestamp: new Date().toISOString()
      });
    }

    return NextResponse.json({
      success: true,
      ...watch,
      timestamp: new Date().toISOString()
    });
  } catch (error) {
    console.error('Agriculture Watch API Error:', error);
    return NextResponse.json(
      { success: false, error: 'Failed to fetch agriculture watch' },
      { status: 500 }
    );
  }
}

```

```

//
=====
====
// 4. CLIMATE FORECAST API
//
=====
====

// app/api/climate/forecast/route.ts

const ForecastQuerySchema = z.object({
  region: z.string().optional(),
  country: z.string().length(2).optional(),
  limit: z.coerce.number().default(5).min(1).max(20),
});

/**
 * GET /api/climate/forecast
 * Get climate forecast from ICPAC
 *
 * Query Parameters:
 * - region: Specific region (eastern, western, northern, southern)
 * - country: 2-letter country code
 * - limit: Maximum number of forecasts
 *
 * Example:
 * /api/climate/forecast
 * /api/climate/forecast?region=eastern
 * /api/climate/forecast?country=KE
 */
export async function GET_ClimateForecast(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = ForecastQuerySchema.safeParse({
    region: searchParams.get('region'),
    country: searchParams.get('country'),
    limit: searchParams.get('limit'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }
}

```



```

    );
  }

  try {
    let forecasts;

    if (query.data.country) {
      // Get forecast for specific country
      const country = IGAD.COUNTRIES[query.data.country as keyof typeof IGAD.COUNTRIES];
      if (!country) {
        return NextResponse.json(
          { success: false, error: 'Invalid country code' },
          { status: 400 }
        );
      }
      forecasts = await DataAggregator.getClimateForecast(country.name);
    } else {
      forecasts = await DataAggregator.getClimateForecast(query.data.region);
    }

    // Apply limit
    if (query.data.limit) {
      forecasts = forecasts.slice(0, query.data.limit);
    }

    return NextResponse.json({
      success: true,
      forecasts,
      count: forecasts.length,
      timestamp: new Date().toISOString()
    });
  } catch (error) {
    console.error('Climate Forecast API Error:', error);
    return NextResponse.json(
      { success: false, error: 'Failed to fetch climate forecast' },
      { status: 500 }
    );
  }
}

//
=====
====
// 5. WEATHER FORECAST API

```

```
//
=====

// app/api/climate/weather/route.ts

const WeatherQuerySchema = z.object({
  latitude: z.coerce.number().optional(),
  longitude: z.coerce.number().optional(),
  country: z.string().length(2).optional(),
  region: z.string().optional(),
  days: z.coerce.number().default(7).min(1).max(14),
});

/**
 * GET /api/climate/weather
 * Get weather forecast from Open-Meteo
 *
 * Query Parameters:
 * - latitude: Latitude coordinate
 * - longitude: Longitude coordinate
 * - country: 2-letter country code (uses capital coordinates)
 * - region: Region name (uses capital coordinates)
 * - days: Number of forecast days (1-14)
 *
 * Example:
 * /api/climate/weather?latitude=-1.2921&longitude=36.8219
 * /api/climate/weather?country=KE
 * /api/climate/weather?country=KE&days=14
 */
export async function GET_Weather(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = WeatherQuerySchema.safeParse({
    latitude: searchParams.get('latitude'),
    longitude: searchParams.get('longitude'),
    country: searchParams.get('country'),
    region: searchParams.get('region'),
    days: searchParams.get('days'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
    );
  }
}
```

```

    { status: 400 }
  );
}

try {
  let lat = query.data.latitude;
  let lng = query.data.longitude;

  // Get coordinates from country or region
  if (query.data.country) {
    const coords = IGAD.COORDS[query.data.country as keyof typeof IGAD.COORDS];
    if (coords) {
      lat = coords[0];
      lng = coords[1];
    }
  }

  // If still no coordinates, use default (Nairobi)
  if (!lat || !lng) {
    lat = IGAD.COORDS.KE[0];
    lng = IGAD.COORDS.KE[1];
  }

  const weather = await DataAggregator.getWeather(lat, lng, query.data.days);

  return NextResponse.json({
    success: true,
    ...weather,
    timestamp: new Date().toISOString()
  });
} catch (error) {
  console.error('Weather API Error:', error);
  return NextResponse.json(
    { success: false, error: 'Failed to fetch weather data' },
    { status: 500 }
  );
}

//
=====
====
// 6. PRODUCTION DATA API

```

```
//
=====

// app/api/production/route.ts

const ProductionQuerySchema = z.object({
  country: z.string().length(2).optional(),
  crop: z.string().optional(),
  year: z.coerce.number().optional(),
  limit: z.coerce.number().default(10).min(1).max(100),
});

/**
 * GET /api/production
 * Get agricultural production data from FAOSTAT
 *
 * Query Parameters:
 * - country: 2-letter country code
 * - crop: Crop code or name
 * - year: Specific year
 * - limit: Maximum number of results
 *
 * Example:
 * /api/production?country=KE
 * /api/production?crop=maize
 * /api/production?country=KE&crop=maize&year=2023
 */
export async function GET_Production(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = ProductionQuerySchema.safeParse({
    country: searchParams.get('country'),
    crop: searchParams.get('crop'),
    year: searchParams.get('year'),
    limit: searchParams.get('limit'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }
}
```

```

try {
  let data;

  if (query.data.country) {
    data = await DataAggregator.getProduction(query.data.country, query.data.crop);
  } else if (query.data.crop) {
    const countries = Object.keys(IGAD.COUNTRIES);
    data = await Promise.all(
      countries.map(c => DataAggregator.getProduction(c, query.data.crop))
    );
    data = data.flat();
  } else {
    const countries = Object.keys(IGAD.COUNTRIES);
    data = await Promise.all(
      countries.map(c => DataAggregator.getProduction(c))
    );
    data = data.flat();
  }

  // Apply limit
  if (query.data.limit && Array.isArray(data)) {
    data = data.slice(0, query.data.limit);
  }

  return NextResponse.json({
    success: true,
    data,
    count: Array.isArray(data) ? data.length : 1,
    timestamp: new Date().toISOString()
  });
} catch (error) {
  console.error('Production API Error:', error);
  return NextResponse.json(
    { success: false, error: 'Failed to fetch production data' },
    { status: 500 }
  );
}

//
=====
=====
// 7. WORLD BANK INDICATORS API

```

```

//
=====

// app/api/indicators/route.ts

const IndicatorsQuerySchema = z.object({
  country: z.string().length(2),
  indicator: z.string().optional(),
  year: z.coerce.number().optional(),
});

/**
 * GET /api/indicators
 * Get World Bank agriculture indicators
 *
 * Query Parameters:
 * - country: 2-letter country code (required)
 * - indicator: Specific indicator code
 * - year: Specific year
 *
 * Example:
 * /api/indicators?country=KE
 * /api/indicators?country=KE&indicator=AG.LND.AGRI.ZS
 */
export async function GET_Indicators(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = IndicatorsQuerySchema.safeParse({
    country: searchParams.get('country'),
    indicator: searchParams.get('indicator'),
    year: searchParams.get('year'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }

  if (!query.data.country) {
    return NextResponse.json(
      { success: false, error: 'Country parameter is required' },

```

```

        { status: 400 }
    );
}

try {
    const indicators = await DataAggregator.getAgricultureIndicators(query.data.country);

    // Filter by indicator if specified
    let data = indicators;
    if (query.data.indicator) {
        data = indicators.filter(i => i.indicatorCode === query.data.indicator);
    }

    // Filter by year if specified
    if (query.data.year) {
        data = data.filter(i => i.year === query.data.year);
    }

    return NextResponse.json({
        success: true,
        data,
        count: data.length,
        country: IGAD.COUNTRIES[query.data.country as keyof typeof IGAD.COUNTRIES]?.name,
        timestamp: new Date().toISOString()
    });
} catch (error) {
    console.error('Indicators API Error:', error);
    return NextResponse.json(
        { success: false, error: 'Failed to fetch indicators' },
        { status: 500 }
    );
}
}

//
=====
====
// 8. ADVISORY API
//
=====
====

// app/api/advisory/route.ts

```

```

const AdvisoryQuerySchema = z.object({
  country: z.string().length(2).optional(),
  crop: z.string().optional(),
  type: z.string().optional(),
  language: z.string().default('en'),
  limit: z.coerce.number().default(10).min(1).max(50),
});

/**
 * GET /api/advisory
 * Get agricultural advisory content
 *
 * Query Parameters:
 * - country: 2-letter country code
 * - crop: Crop name
 * - type: Advisory type (planting, harvesting, fertilizer, pest_control)
 * - language: Response language (en, sw, am, ar, etc.)
 * - limit: Maximum number of results
 *
 * Example:
 * /api/advisory?country=KE&crop=maize
 * /api/advisory?type=planting
 */
export async function GET_Advisory(request: Request) {
  const { searchParams } = new URL(request.url);

  const query = AdvisoryQuerySchema.safeParse({
    country: searchParams.get('country'),
    crop: searchParams.get('crop'),
    type: searchParams.get('type'),
    language: searchParams.get('language'),
    limit: searchParams.get('limit'),
  });

  if (!query.success) {
    return NextResponse.json(
      { success: false, error: 'Invalid query parameters', details: query.error },
      { status: 400 }
    );
  }

  try {
    // In production, this would query the database
    // For now, return mock advisory content

```



```

const advisory = getAdvisoryContent(
  query.data.country,
  query.data.crop,
  query.data.type,
  query.data.language
);

// Apply limit
if (query.data.limit && Array.isArray(advisory)) {
  advisory.slice(0, query.data.limit);
}

return NextResponse.json({
  success: true,
  advisory,
  count: Array.isArray(advisory) ? advisory.length : 1,
  timestamp: new Date().toISOString()
});
} catch (error) {
  console.error('Advisory API Error:', error);
  return NextResponse.json(
    { success: false, error: 'Failed to fetch advisory content' },
    { status: 500 }
  );
}
}

function getAdvisoryContent(
  countryCode?: string,
  crop?: string,
  type?: string,
  language: string = 'en'
){
  // Mock adviso

```